

Claude Code Deep Dive Workshop

Chapter 2 - Agents (Subagents)

맞춤 서브에이전트로 작업을 분담하고 병렬화하기

Choi, WooHyung

Prin. Solutions Architect

AWS Korea

whchoi@amazon.com



Agenda

Workshop / 6 Chapters across 3 Days

01

Subagents 개념과 원리

메인 자식 관계, Context 격리, 위임의 가치

02

Subagent 정의 방법

.claude/agents, YAML frontmatter, 시스템 프롬프트

03

Task 도구와 디스패치

자동 수동 호출, 병렬 vs 직렬, 결과 처리

04

Pattern 1 / Code Reviewer

PR 자동 리뷰 에이전트 구현

05

Pattern 2 / Tester

커버리지 확장 에이전트 구현

06

Pattern 3-5 / 추가 패턴

Security, Docs, Migration 에이전트

07

Recap & 4 Labs

핵심 정리와 4가지 실습 랩

CHAPTER 02

Agents

Subagents 시스템과 실전 패턴



Learning Objectives

이번 챕터에서 무엇을 배우게 되는가

OBJ 1

Subagents 이해

메인 에이전트와 자식 에이전트의 관계, Context 격리, 위임의 가치를 학습합니다.

OBJ 2

맞춤 에이전트 정의

`.claude/agents` 디렉토리에 YAML frontmatter와 시스템 프롬프트로 에이전트를 작성합니다.

OBJ 3

Task 도구 활용

Task 도구로 자동 또는 수동 디스패치를 수행하고 결과를 후처리합니다.

OBJ 4

실전 패턴 5선

Code Reviewer, Tester, Security Scanner, Docs Writer, Migration 패턴을 구현합니다.

OUTCOME

챕터를 마치면 본인 프로젝트에 맞춤 전문 서브에이전트를 작성하고 Task 도구로 자동 디스패치할 수 있게 됩니다.

4

학습 목표

5

실전 패턴

4

실습 랩

Production

실무 적용 수준

CHAPTER 02 – PART 01

Agents – Subagent

Subagents 시스템과 실전 패턴



왜 Subagents인가

단일 에이전트의 한계와 분담의 가치

PAIN 1

Context 폭발

큰 작업을 한 세션에서 처리하면 200K 토큰이 빠르게 소진되고 정확도가 떨어짐.

PAIN 2

단일 시점 처리

리뷰, 테스트, 보안 검사 같은 다양한 관점을 동시에 다루기 어려움.

PAIN 3

비용 비효율

단순 작업까지 큰 모델로 처리하면 비용이 누적되어 부담이 커짐.

PAIN 4

재사용성 부족

같은 패턴의 작업을 매번 처음부터 프롬프트로 설명해야 함.

THE ANSWER

Subagents는 작업을 전문 에이전트들에게 분담시켜 Context를 격리하고, 적절한 모델을 선택하며, 재사용 가능한 자산으로 축적합니다.

Isolated

Context 격리

Parallel

병렬 처리

Right-sized

적절한 모델 선택

Reusable

재사용 가능 자산

Part 1 / Subagents 개념과 원리

왜 분담하는가, 무엇이 이득인가

PART OBJECTIVE

Subagent의 정확한 정의, 메인 에이전트와의 관계, Context 격리의 의미, 그리고 위임이 가져다주는 4가지 가치를 학습.

TOPIC 1

정의와 관계

Subagent의 공식 정의, 메인 에이전트와 자식 에이전트의 위계 구조를 다룹니다.

TOPIC 2

Context 격리

각 Subagent가 독립된 컨텍스트 윈도우를 갖는 이유와 그 효과를 설명합니다.

TOPIC 3

도구와 모델

Subagent별 도구 권한 격리와 모델 선택의 자유도를 살펴봅니다.

TOPIC 4

실전 사용 사례

6가지 일반적인 사용 사례와 안티 패턴을 함께 다룹니다.

15 Slides

Part 1 분량

~30 min

예상 강의 시간

4 Topics

주요 토픽

Conceptual

개념 중심

Subagent란?

공식 정의와 핵심 특성

DEFINITION

Subagent는 메인 에이전트(Claude Code 세션)가 위임할 수 있는 독립적인 자식 에이전트.
자신만의 시스템 프롬프트, 컨텍스트, 도구 권한, 모델을 소유.

TRAIT 1

독립 컨텍스트

자신만의 200K 토큰 윈도우를 가지며 메인과 격리되어 동작.

TRAIT 2

전용 시스템 프롬프트

역할과 행동 규칙을 명시한 system prompt가 매 호출마다 적용.

TRAIT 3

도구 권한 제한

필요한 도구만 화이트리스트로 선언하여 최소 권한 원칙을 적용.

TRAIT 4

모델 선택 자유

작업 복잡도에 맞춰 Sonnet, Opus, Haiku 중 적절한 모델을 지정.

Isolated

컨텍스트 격리

Specialized

역할 전문화

Scoped

권한 제한

Tunable

모델 튜닝 가능

왜 위임하는가

4가지 핵심 동기

WHY 1

맥락 절약

메인 세션의 컨텍스트를 보존하면서 큰 하위 작업을 별도의 컨텍스트에서 처리.

WHY 2

병렬 처리

여러 Subagent를 동시에 실행하여 처리 시간을 단축.

WHY 3

관심사 분리

리뷰, 테스트, 보안 같은 독립된 관점을 전문 에이전트로 분리.

WHY 4

재사용 자산화

한 번 정의한 에이전트를 팀 전체와 모든 프로젝트에서 재사용.

KEY INSIGHT

위임은 단순한 기능이 아닙니다. AI 워크플로 설계의 핵심 디자인 패턴이며, 확장 가능한 자동화의 기반이 됩니다.

Save context

메인 보호

Parallelize

동시 실행

Separate

관점 분리

Reuse

재사용성

Main Agent vs Subagent

두 에이전트의 역할 차이

MAIN AGENT

메인 에이전트

1. 사용자와 직접 대화
2. 작업의 큰 그림 설계
3. Subagent 디스패치 결정
4. 결과 통합과 사용자 보고
5. 전체 세션 흐름 통제
6. 컨텍스트가 사용자 대화 중심

SUBAGENT

서브 에이전트

1. 메인의 위임으로만 실행
2. 단일 책임에 집중
3. 사용자와 직접 대화 없음
4. 결과를 메인에 반환
5. 격리된 컨텍스트에서 동작
6. 시스템 프롬프트 중심 동작

User-facing

메인의 역할

Task-focused

서브의 역할

1:N

메인:서브 관계

Isolated

컨텍스트

Context Isolation

왜 컨텍스트를 분리하는가

Context Model

Main Agent Context (200K tokens)

User: 이 코드베이스 보안 취약점을 점검해 줘
Main: 분석을 시작하겠습니다.
Main: → Task(security-scanner, ...)

Subagent Context (별도 200K)

- 시스템 프롬프트 (security 전문)
- 파일 100개 분석 (수십만 토큰)
- 분석 종료 후 요약만 반환

Main: 결과 요약: 5건의 high severity 발견.
User: 첫 번째 항목을 자세히 설명해 줘

200K + 200K

독립 윈도우

Summary

요약만 반환

No leak

메인 보호

Reusable

메인 재활용

위임의 가치

구체적인 이득 4가지

Aspect	단일 에이전트 방식	Subagent 위임 방식
Context	모든 정보가 메인에 누적	메인 보존, 서브에서 격리 처리
Time	순차 실행만 가능	병렬 실행으로 단축
Cost	모든 작업이 같은 모델 사용	작업별 최적 모델 선택
Quality	범용 프롬프트로 모든 일 처리	전문 프롬프트로 깊이 있는 결과
Reuse	매번 처음부터 지시 필요	한 번 정의로 전팀이 재사용

4x
병렬화 시 처리량

60-80%
비용 절감 가능

Deeper
전문 결과 깊이

Asset
팀 자산화

Subagent의 상태 모델

Stateless 호출의 의미

STATELESS DESIGN

Subagent는 기본적으로 stateless입니다. 매 호출이 독립적이며 이전 호출의 상태를 기억하지 않습니다. 메인이 필요한 컨텍스트를 매번 입력으로 전달합니다.

STATELESS

기본 동작

1. 매 호출이 독립 세션
2. 이전 호출 결과 기억 없음
3. 메인이 필요 정보 매번 전달
4. 멍등성 보장
5. 병렬 호출 안전

STATEFUL

필요 시 우회

1. 파일에 상태 저장
2. CLAUDE.md 또는 데이터 파일
3. 다음 호출에서 다시 읽음
4. 진정한 stateful은 불가
5. 자동화 시 신중히 설계

Stateless

기본 설계 원칙

Idempotent

멍등 호출

File-based

상태 우회 방식

Parallel-safe

병렬 안전

도구 권한 격리

Subagent별 최소 권한 적용

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: PR diff를 검토하고 개선 사항을 제안
tools: Read, Grep, Glob, Bash(git diff:*)
model: claude-sonnet-4-5
---
```

tools field

당신은 시니어 코드 리뷰어입니다.
주어진 PR diff를 다음 관점에서 검토하세요:

- 가독성과 명명 규칙
- 잠재적 버그와 엣지 케이스
- 성능 영향
- 테스트 커버리지

tools:

도구 화이트리스트

Glob

도구 이름

Bash(git:*)

명령 패턴

Least-priv

최소 권한

모델 선택

Subagent별 적절한 모델 지정

Model	추천 사용처	비용 특성
claude-opus-4-5	복잡한 설계, 심층 추론, 보안 분석	가장 비쌈, 가장 정확
claude-sonnet-4-5	일반 코딩, 리뷰, 리팩토링 (기본)	균형, 표준 선택
claude-haiku-4-5	단순 분류, 추출, 포맷 변환	가장 저렴, 빠름
명시 안 함	메인이 자동 라우팅	Sonnet 기본

GUIDANCE

단순 작업은 Haiku로 위임해 비용을 절감. 깊은 분석이 필요한 보안이나 설계 작업은 Opus로 위임. 대부분의 일상 작업은 Sonnet으로 충분.

Opus

심층 추론

Sonnet

범용 기본

Haiku

경량 빠름

Auto

메인 자동 선택

Subagent 6대 사용 사례

실무에서 가장 흔한 패턴

Pattern	용도	모델
Code Reviewer	PR diff 자동 리뷰, 개선 제안	Sonnet
Tester	테스트 커버리지 확장, 엡지 케이스 발굴	Sonnet
Security Scanner	취약점, 의존성, 시크릿 검사	Opus
Docs Writer	README, API 문서, ADR 자동 작성	Sonnet
Migration Bot	대량 파일 일괄 변경 (lib upgrade 등)	Haiku
Research Agent	코드베이스 학습, 의존성 매핑	Sonnet

6

대표 패턴

3

활용 모델

Production

실무 검증됨

Composable

조합 가능

Built-in Agents

Claude Code 기본 제공 에이전트

GENERAL

general-purpose

범용 위임 에이전트. 메인이 명시적 지정 없이 위임할 때 기본으로 호출.

EXPLORE

codebase-explorer

대용량 코드베이스 탐색 전용.
Glob/Grep으로 구조와 의존성을 매핑.

CUSTOM

사용자 정의

본인이 .claude/agents에 작성한 모든 맞춤 에이전트도 자동으로 후보가 됨.

NOTE

Built-in 에이전트는 별도 정의 없이 즉시 사용할 수 있으며, 맞춤 에이전트가 description으로 더 잘 매칭되면 그쪽이 우선 선택.

general

범용 위임

explorer

코드 탐색

custom

사용자 정의

auto

자동 매칭

일반 에이전트 vs 전문 에이전트

언제 무엇을 선택하는가

GENERAL

일반 에이전트

1. 다양한 작업을 처리 가능
2. 시스템 프롬프트가 광범위
3. 처음 도입할 때 편함
4. 깊이 있는 결과는 어려움
5. 적합: 탐색, 초기 분석

SPECIALIZED

전문 에이전트

1. 단일 책임에 집중
2. 시스템 프롬프트가 명확
3. 좁고 깊은 작업에 강함
4. 재사용성이 높음
5. 적합: 반복 작업, 품질 중심

Broad

일반의 강점

Deep

전문의 강점

Mix

실무는 혼합

Evolve

점진 전환

Cost & Latency 고려사항

병렬화의 진짜 비용

Cost Calc

단일 에이전트
보안 검사 (Sonnet) → 50K input + 5K output → 약 \$0.23, 60초

Subagent 위임 시 (3개 병렬)

Main (Sonnet)

└ security-scanner (Opus)	50K + 5K → \$1.13, 80초
└ docs-writer (Sonnet)	30K + 8K → \$0.21, 40초
└ test-extender (Sonnet)	40K + 10K → \$0.27, 50초

총 비용: \$1.61 (vs 단일 \$0.23, 약 7배)

총 시간: $\max(80, 40, 50) = 80$ 초 (병렬, vs 순차 170초)

트레이드오프

+ 시간 단축 (170s → 80s, -53%)

+ 작업별 모델 최적화 가능

- 비용 증가 (7배)

- 메인 컨텍스트 보존 가치 별도

-53%

병렬 시 시간 단축

+7x

비용 증가 가능

Opus

비용 주범

Tune

모델 선택으로 조정

Subagent 안티 패턴

피해야 할 5가지 실수

AVOID 1

과도한 분할

모든 작은 작업까지 위임하면 오버헤드가 이득을 초과.

AVOID 2

권한 화이트리스트 누락

tools 필드 없이 모든 도구 허용은 위험.
반드시 명시.

AVOID 3

모호한 description

description이 모호하면 자동 디스패치가 잘못된 에이전트로 감.

AVOID 4

Opus 남발

단순 작업까지 Opus로 위임하면 비용이 빠르게 누적.

AVOID 5 / Stateful 의존

Subagent는 stateless가 기본. 호출 간 상태 공유가 필요한 작업은 메인에서 처리하거나 파일 기반으로 명시적 설계가 필요.

Right-size

분할 단위

Tools

권한 명시

Clear

description

Stateless

기본 가정

Part 1 Recap

개념 핵심 정리

TAKEAWAY 1

독립 격리 자원

Subagent는 자신만의 컨텍스트, 도구, 모델을 가진 독립 에이전트

TAKEAWAY 2

4대 가치

Context 절약, 병렬 처리, 관심사 분리, 재사용 자산화

TAKEAWAY 3

6대 사용 패턴

Reviewer, Tester, Security, Docs, Migration, Research

TAKEAWAY 4

신중한 트레이드오프

시간 절감과 비용 증가 사이의 균형, 적절한 모델 선택 필수

NEXT - PART 2

Part 2는 Subagent를 실제로 정의하는 방법. .claude/agents 디렉토리 구조, YAML frontmatter 필드, 시스템 프롬프트 작성법을 다룸.

Part 1

완료

Part 2

Definition (15p)

Part 3

Task tool (15p)

~3.5h

잔여 시간

CHAPTER 02 – PART 02

Agents – Subagent 정의 방법

Subagents 시스템과 실전 패턴



Part 2 / Subagent 정의 방법

실제로 어떻게 만드는가

PART OBJECTIVE

.claude/agents 디렉토리에 Subagent를 작성하는 방법을 단계별로 학습.

YAML frontmatter 모든 필드와 시스템 프롬프트 작성 기법을 다룹니다.

TOPIC 1

디렉토리 구조

.claude/agents/와
~/.claude/agents/ 두 위치
와 우선순위 규칙을 다룹니다.

TOPIC 2

YAML frontmatter

name, description,
tools, model 네 가지 필드
의 작성법을 익힙니다.

TOPIC 3

시스템 프롬프트

역할, 행동 규칙, 출력 형식
을 명확히 작성하는 패턴을
학습합니다.

TOPIC 4

실전 예시

Code Reviewer, Tester,
Docs Writer 3가지 완성
예시를 함께 살펴봅니다.

15 Slides

Part 2 분량

~30 min

예상 강의 시간

4 Fields

frontmatter 필드

3 Examples

완성 예시

.claude/agents 디렉토리

Subagent 파일이 사는 곳

Directory

```
# 1. 프로젝트 수준 (팀 공유, Git 커밋 대상)
your-project/
├── .claude/
│   ├── agents/
│   │   ├── code-reviewer.md    ← Subagent 정의 파일
│   │   ├── test-writer.md
│   │   ├── docs-writer.md
│   │   └── security-scanner.md
│   ├── settings.json
│   └── CLAUDE.md
└── src/

# 2. 사용자 수준 (개인 전역, 모든 프로젝트에서 사용)
~/.claude/
└── agents/
    ├── my-personal-helper.md
    └── language-translator.md

# 3. 파일 형식: 항상 .md (Markdown with YAML frontmatter)
```

.claude/agents/

프로젝트 단위

~/.claude/agents/

개인 전역

.md

파일 확장자

Git

버전 관리

우선순위 규칙

같은 이름이 충돌할 때

STEP 01

프로젝트
.`claude/agents/`

STEP 02

사용자
~/.`claude/agents/`

STEP 03

Built-in
시스템 제공

STEP 04

디스패치
최종 결정

RULE

동일한 name의 에이전트가 여러 위치에 있으면 프로젝트 수준이 가장 우선. 그 다음 사용자 수준, 마지막으로 Built-in.

PRACTICE

팀 표준 에이전트는 프로젝트에, 개인 도우미는 사용자 디렉토리에 둬. 두 위치는 서로 보완 관계.

Project

최우선

User

두 번째

Built-in

기본 fallback

Override

상위가 덮어씀

YAML Frontmatter 구조

Subagent 정의 파일의 윗부분

YAML frontmatter

```
# .claude/agents/example.md
```

```
---
```

```
name: code-reviewer
```

```
description: PR diff를 검토하고 개선 사항을 제안하는 시니어 리뷰어
```

```
tools: Read, Grep, Glob, Bash(git diff:*)
```

```
model: claude-sonnet-4-5
```

```
---
```

여기서부터는 시스템 프롬프트 본문입니다.

당신은 시니어 코드 리뷰어입니다.

주어진 코드 변경을 다음 관점에서 검토하세요.

- 가독성
- 안전성
- 성능
- 테스트 커버리지

frontmatter 구분자

4 fields

핵심 필드 수

name

필수 필드

description

필수 필드

name 필드

에이전트의 고유 식별자

GOOD

좋은 이름 예시

- code-reviewer
 - test-writer
 - security-scanner
 - docs-writer
 - migration-bot
 - db-schema-designer
- kebab-case, 역할이 명확

AVOID

피해야 할 이름

- helper (모호함)
 - Agent1 (의미 없음)
 - MyAgent (개인적)
 - 한글이름 (호환성 문제)
 - code_reviewer (snake_case)
 - CodeReviewer (PascalCase)
- 모호하거나 비표준

kebab-case

명명 규칙

Role-based

역할 기반

Lower-only

소문자만

ASCII

영문자만

description 필드

자동 디스패치의 핵심

WHY CRITICAL

메인 에이전트는 사용자 요청을 받으면 description을 보고 어떤 Subagent를 호출할지 자동 결정.

모호한 description은 잘못된 디스패치를 유발.

```
# 좋은 description (구체적, 작동 시점이 명확)
description: |
  PR diff를 검토하고 가독성, 안전성, 성능 관점에서 개선 사항을 제안.
  사용 시점: 코드 리뷰가 필요하다고 사용자가 명시하거나 git diff 검토 시.
```

```
# 나쁜 description (모호함)
description: 코드를 분석하는 도우미
```

```
# 더 나쁜 description (너무 광범위)
description: 모든 개발 작업을 도와주는 만능 에이전트
```

description

Concrete

구체적

When-to-use

사용 시점 명시

Scoped

범위 좁힘

Routable

자동 디스패치 가능

tools 필드

도구 화이트리스트 작성법

tools patterns

1. 도구 이름만 (해당 도구 전체 허용)

tools: Read, Grep, Glob, Edit, Write

2. Bash는 명령 패턴으로 좁히기

tools: Read, Bash(npm test:*), Bash(git diff:*), Bash(git status)

3. 패턴 와일드카드

tools: Read(**/*.md), Edit(docs/**)

4. 비어 있으면 → 메인의 모든 도구 상속 (지양)

tools 필드 생략 시 위험

5. 빈 문자열 → 도구 없음 (분석 전용 에이전트)

tools: ""

Whitelist

명시적 허용

Bash(*)

명령 패턴

Read(**)

경로 그룹

""

도구 없음

model 필드

모델 선택 명시

값	의미	권장 사용
claude-opus-4-7	최강 모델, 심층 추론	복잡한 보안, 설계
claude-sonnet-4-6	균형 모델, 표준 선택	대부분의 일상 작업
claude-haiku-4-5	경량 모델, 빠르고 저렴	단순 분류, 추출
inherit	메인의 모델 상속	메인 결정에 위임
생략	자동 라우팅 (보통 Sonnet)	특별한 사유 없을 때

BUDGET TIP

Migration이나 단순 추출 같은 반복 작업은 Haiku로 명시 지정하면 비용을 크게 절감. Opus는 정말 필요한 작업에만 사용.

opus

심층 분석

sonnet

범용 기본

haiku

경량 빠름

inherit

메인 상속

System Prompt 작성법

본문에 어떤 내용을 담는가

SECTION 1

역할 선언

한 문장으로 에이전트의 정체성과 책임을 명확히 정의.

SECTION 2

작업 규칙

에이전트가 따라야 할 구체적인 행동 규칙과 우선순위를 나열.

SECTION 3

출력 형식

메인에 어떤 형식으로 결과를 반환할지 명시.

SECTION 4

예외 처리

판단이 어려운 상황과 처리 방법을 미리 정해둌.

PRINCIPLE

시스템 프롬프트는 사람에게 일을 시키는 명세서처럼 작성. 한국어와 영어를 자연스럽게 섞어 쓸 수 있으며, 명확함이 가장 중요.

Role

정체성

Rules

행동 규칙

Format

출력 형식

Edge

예외 처리

description 작성 비교

구체성이 디스패치 정확도를 결정

comparison

시나리오: 사용자가 "이 PR의 보안 이슈를 점검해 줘" 요청

❌ 모호한 description (잘못된 디스패치 위험)

agent A: description: 코드 관련 분석을 수행합니다.

agent B: description: 코드 검사를 도와줍니다.

→ 메인이 어떤 걸 골라야 할지 모름

✅ 명확한 description

agent A: description: |

PR diff를 가독성, 명명 규칙, 잠재 버그 관점에서 검토.

사용 시점: 일반 코드 리뷰가 필요한 경우.

agent B: description: |

코드와 의존성을 보안 취약점(injection, XSS, secrets) 관점에서 분석.

사용 시점: 보안 이슈 점검이 필요한 경우.

→ 메인이 보안 키워드를 보고 agent B를 정확히 선택

Specific

구체적 단어

When

사용 시점 명시

Boundary

경계 명확

Routable

디스패치 가능

완성 예시 1 / code-reviewer

실무에서 바로 쓸 수 있는 정의

code-reviewer.md

```
---
name: code-reviewer
description: |
  PR diff를 검토하여 가독성, 안전성, 성능, 테스트 커버리지 관점에서
  개선 사항을 제안하는 시니어 코드 리뷰어. 사용 시점: 코드 리뷰가
  필요하다고 사용자가 요청하거나 git diff 검토가 필요한 경우.
tools: Read, Grep, Glob, Bash(git diff:*), Bash(git log:*)
model: claude-sonnet-4-5
---
```

당신은 10년 경력의 시니어 소프트웨어 엔지니어입니다.

검토 우선순위

1. 안전성: null 참조, race condition, 입력 검증 누락
2. 가독성: 명명, 함수 크기, 주석 필요성
3. 성능: 알고리즘 복잡도, 불필요한 I/O
4. 테스트: 새 코드의 테스트 커버리지

출력 형식

- Severity: high | medium | low
- 파일:라인, 문제 설명, 권장 수정안

Sonnet

균형 모델

Read-only

안전한 권한

4 priorities

명확한 우선순위

Structured

구조화 출력

완성 예시 2 / test-writer

커버리지 확장 전문 에이전트

test-writer.md

```
---
name: test-writer
description: |
  기존 코드의 테스트 커버리지를 분석하고 누락된 엡지 케이스, 분기,
  예외 경로에 대한 테스트를 작성. 사용 시점: 커버리지 부족이 확인되거나
  새 함수에 테스트가 필요한 경우.
tools: Read, Grep, Glob, Edit, Write, Bash(npm test:*), Bash(pytest:*)
model: claude-sonnet-4-5
---
```

당신은 테스트 자동화 전문가입니다.

작업 절차

1. 대상 함수의 시그니처와 분기 구조 분석
2. 기존 테스트 파일이 있다면 스타일 일치
3. 정상 케이스, 엡지 케이스, 예외 케이스 모두 커버
4. 테스트 작성 후 실제로 실행하여 통과 확인

작성 원칙

- 테스트 하나가 한 가지만 검증
- 의미 있는 테스트 이름
- AAA 패턴 (Arrange Act Assert)

Sonnet

코드 작성용

Edit/Write

테스트 생성

AAA

테스트 패턴

Verify

실행 검증

완성 예시 3 / docs-writer

문서 자동 생성 에이전트

docs-writer.md

```
---
name: docs-writer
description: |
  README, API 문서, 아키텍처 결정 문서(ADR)를 코드와 일치하도록 작성하거나
  업데이트. 사용 시점: 신규 기능 추가 후 문서 작성이 필요하거나 기존 문서가
  코드와 불일치할 때.
tools: Read, Grep, Glob, Edit, Write, Bash(git log:*)
model: claude-sonnet-4-5
---
```

당신은 기술 문서 작성가입니다.

문서 종류별 가이드

- README: 프로젝트 개요, 설치, 사용법, 예시
- API 문서: 엔드포인트, 파라미터, 응답 스키마, 예시
- ADR: Context, Decision, Consequences 형식

작성 원칙

- 코드와 실제 동작이 일치해야 함
- 예시는 실제로 실행 가능한 것
- 한국어와 영어 혼용 가능, 일관성 유지

3 types

README/API/ADR

Sync

코드와 일치

Runnable

실행 가능 예시

Bilingual

한영 혼용 가능

권한 모델 통합

settings.json과의 관계

Permission Model

```
# 1. settings.json (프로젝트 전역 권한)
{
  "permissions": {
    "allow": ["Read(**)", "Bash(npm test:*)"],
    "ask": ["Edit(**)", "Write(**)"],
    "deny": ["Bash(rm -rf:*)"]
  }
}

# 2. Subagent의 tools 필드 (해당 에이전트만의 제한)
tools: Read, Edit, Bash(npm test:*)

# 3. 실제 적용 (교집합)
- Read      → allow (자동 허용)
- Edit      → ask  (사용자 확인 필요)
- Bash(npm) → allow
- Bash(rm)  → deny (Subagent가 시도해도 차단)
```

Settings

프로젝트 정책

tools

Subagent 줍힘

Intersect

교집합 적용

Deny wins

deny가 항상 우선

Part 2 Recap

정의 방법 핵심 정리

TAKEAWAY 1

두 위치

프로젝트(.claude/agents)는
팀 공유, 사용자(~)는 개인 자산

TAKEAWAY 2

4 필드

name(식별), description(자
동 디스패치), tools(권한),
model(비용)

TAKEAWAY 3

시스템 프롬프트

역할, 행동 규칙, 출력 형식, 예
외 처리 4섹션 구성

TAKEAWAY 4

권한 통합

tools와 settings.json은 교집
합으로 적용, deny는 항상 우선

NEXT - PART 3

Part 3은 Task 도구를 통한 디스패치 메커니즘입니다. 자동/수동 호출, 병렬 vs 직렬, 결과 처리 패턴을 15장에 걸쳐 다룹니다.

Part 2

완료

Part 3

Task tool (15p)

Part 4-8

Patterns (50p)

~3h

잔여 시간

CHAPTER 02 - PART 03

Agents – Task 도구와 디스패치

Subagents 시스템과 실전 패턴



Part 3 / Task 도구와 디스패치

Subagent를 어떻게 호출하는가

PART OBJECTIVE

Task 도구를 통해 Subagent를 호출하는 메커니즘을 학습합니다. 자동 디스패치와 명시적 호출, 병렬 실행, 결과 처리, 비용 추적까지 다룹니다.

TOPIC 1

도구 구조

Task 도구의 입력 스키마와 호출 시 메인이 무엇을 결정하는지 다룹니다.

TOPIC 2

디스패치 방식

자동 디스패치와 명시적 호출 두 가지 방식의 차이와 선택 기준입니다.

TOPIC 3

병렬과 직렬

여러 Subagent를 동시 또는 순차로 실행하는 방법과 트레이드오프입니다.

TOPIC 4

결과 처리

반환된 결과의 후처리, 에러 처리, 비용 추적 기법을 학습합니다.

15 Slides

Part 3 분량

~30 min

예상 강의 시간

Mechanism

메커니즘 중심

Code-heavy

코드 예시 풍부

Task 도구 개요

Subagent 호출의 단일 진입점

TASK TOOL

Task는 메인 에이전트가 Subagent를 호출하는 유일한 메커니즘. 호출할 Subagent의 이름, 작업 설명, 컨텍스트를 함께 전달.

```
# Task 호출 형식 (메인의 tool_use 블록)
Task(
    subagent_type="code-reviewer",
    description="Review the recent commit",
    prompt="""
최근 커밋 ${SHA}의 변경사항을 검토해 주세요.
특히 인증 관련 변경에 주목하세요.

관련 파일:
- src/auth/middleware.js
- src/auth/jwt.js
"""
)
# 반환 형식 (Subagent의 결과)
{ "summary": "...", "issues": [...], "tokens_used": 5234 }
```

Task call

subagent_type

호출 대상

description

작업 라벨

prompt

실제 지시

Return

결과 반환

자동 디스패치

메인이 알아서 Subagent를 고르는 메커니즘

STEP 01

사용자
요청 수신

STEP 02

description
전체 검색

STEP 03

가장 적합한
Subagent 선택

STEP 04

Task 호출
자동 실행

```
# 사용자 요청
> 최근 PR의 보안 이슈를 점검해 줘.
# 메인의 자동 디스패치 (사용자가 별도 지정 안 함)
- 모든 Subagent의 description 검토
- "보안" 키워드가 description에 있는 security-scanner 선택
- Task(subagent_type="security-scanner", ...)
- 사용자는 어떤 에이전트가 호출되었는지 결과와 함께 안내받음
# 잘 작동하려면
- description이 구체적이고 사용 시점 명시
- 에이전트끼리 description 영역이 명확히 구분됨
```

auto dispatch

Implicit

암묵적 호출

Description

매칭 기준

Best-match

최적 선택

Transparent

사용자에 안내

명시적 호출

사용자가 직접 에이전트를 지정

explicit calls

```
# 방법 1: 자연어로 지정
> code-reviewer 에이전트를 사용해서 이 PR을 검토해 줘.

> security-scanner로 의존성 취약점을 검사해 줘.

# 방법 2: /agents 슬래시 명령으로 목록 확인 후 선택
> /agents
Available agents:
- code-reviewer
- test-writer
- security-scanner
- docs-writer

> code-reviewer 사용

# 방법 3: 메인 프롬프트에서 직접 지시
> [Use code-reviewer subagent]
> 최근 commit을 리뷰해 줘.
```

By name

이름 명시

/agents

목록 확인

Override

자동 디스패치 우선

Predictable

예측 가능

병렬 VS 직렬 실행

여러 Subagent의 호출 패턴

PARALLEL

병렬 실행

1. 여러 Task 호출을 동시에 실행
2. 서로 독립적인 작업에 적합
3. 시간 단축이 가장 큰 이득
4. 비용은 합산, 시간은 max
5. 예시: 리뷰 + 보안 + 테스트 동시

SERIAL

직렬 실행

1. 한 Task가 끝난 후 다음 호출
2. 결과 의존성이 있는 작업에 적합
3. 메인이 중간 결정을 반영
4. 비용과 시간 모두 합산
5. 예시: 분석 → 그 결과로 다음 분석

Time-save

병렬의 장점

Dependent

직렬의 필요

Cost

둘 다 합산

Hybrid

혼합도 가능

병렬 호출 예시

한 번에 3개 Subagent 동시 실행

Parallel call

```
# 사용자 요청
> 이 PR을 전반적으로 점검해 줘. 리뷰, 보안, 테스트 추가까지.

# 메인의 응답 (3개 Task를 한 번에 발행)
Task(subagent_type="code-reviewer",    prompt="...PR diff...")
Task(subagent_type="security-scanner",  prompt="...PR files...")
Task(subagent_type="test-writer",      prompt="...changed funcs...")

# 실행 (병렬)
code-reviewer:    45초 → 8개 이슈 발견
security-scanner: 60초 → 2개 high severity 발견
test-writer:      50초 → 4개 테스트 추가

총 소요 시간: max(45, 60, 50) = 60초

# 메인이 3개 결과를 통합해 사용자에게 보고
"전체 검토 완료: 코드 이슈 8건, 보안 이슈 2건, 신규 테스트 4개."
```

3

동시 실행 수

60s

병렬 총 시간

vs 155s

직렬이라면

-61%

시간 절감

결과 처리 패턴

Subagent 결과를 메인에서 활용

Pattern	메인의 동작	사용 사례
Summary	결과를 그대로 사용자에게 전달	단순 보고형 작업
Aggregate	여러 Subagent 결과를 집계	병렬 다중 호출
Conditional	결과 내용에 따라 다음 호출 결정	동적 워크플로
Action	결과 기반으로 코드 수정이나 액션 실행	리뷰 → 자동 fix
Persist	결과를 파일이나 메모리에 저장	CLAUDE.md 갱신

BEST PRACTICE

Subagent 결과는 가급적 구조화된 형식(JSON, 마크다운 섹션)으로 반환하도록 시스템 프롬프트에서 지시.(후처리가 훨씬 쉬워집니다.)

Structured

구조화 권장

Aggregate

집계 패턴

Conditional

조건 분기

Persist

영구 저장

컨텍스트 전달

Subagent에 무엇을 함께 보내는가

context pass

```
# Subagent는 메인 컨텍스트를 자동으로 받지 않음
# 필요한 정보를 prompt에 명시적으로 담아 전달해야 함
```

```
# 좋은 예시 (충분한 컨텍스트)
```

```
Task(
    subagent_type="code-reviewer",
    prompt=f"""
작업: 다음 PR diff를 검토해 주세요.
```

```
변경 요약: 사용자 인증 미들웨어에 JWT 검증 추가
관련 파일: src/auth/middleware.js, tests/auth.test.js
```

```
PR diff:
${diff_content}
```

```
특별 주의 사항: HS256 vs RS256 알고리즘 선택 적절성
"""
```

```
)
```

```
# 피해야 할 예시 (정보 부족)
```

```
Task(subagent_type="code-reviewer", prompt="PR 리뷰해 줘")
```

Explicit

명시 전달

No auto

자동 상속 없음

Files

관련 파일 명시

Goal

목표 명확화

에러 처리

Subagent 실패에 대한 대응

CASE 1

권한 거부

Subagent가 tools 외 호출 시도.
settings.json의 deny 적용.
메인은 실패 안내.

CASE 2

타임아웃

장시간 실행 시 자동 중단. 메인은 부분 결과 또는 재시도 결정.

CASE 3

컨텍스트 초과

Subagent의 200K 초과.
메인은 입력 분할 후 재호출하거나 다른 모델 선택.

CASE 4

결과 형식 위반

Subagent가 요청 형식대로 반환 안 함.
메인이 재시도하거나 부분 사용.

GENERAL APPROACH

메인은 Subagent의 실패를 사용자에게 투명하게 보고합니다. 자동 재시도는 보통 1회로 제한하고, 반복 실패 시 사용자 개입을 요청.

Transparent

사용자에 보고

1 retry

재시도 횟수

Fail-fast

빠른 실패

Manual

복잡 시 수동

Cost Tracking

Subagent 사용 비용 모니터링

```
# /cost 명령으로 현재 세션 비용 확인
> /cost
Current session cost: $2.34
```

Breakdown:

Main agent (Sonnet):	\$0.45 (12K in, 8K out)
code-reviewer (Sonnet):	\$0.32 (45K in, 5K out)
security-scanner (Opus):	\$1.41 (38K in, 6K out)
test-writer (Sonnet):	\$0.16 (22K in, 4K out)

Total tokens: 153K

Total time: 4m 32s

```
# 자동 알람 설정 (settings.json)
{
  "cost_alerts": {
    "session_max": 5.00,
    "daily_max": 50.00
  }
}
```

/cost

/cost

확인 명령

Per-agent

Subagent별 집계

Alerts

알람 설정

Daily

일별 한도

Subagent 디버깅

예상과 다른 동작 파악법

증상	원인 후보	해결
예상과 다른 에이전트 호출	description 모호함	명시적 호출 또는 description 개선
결과가 너무 짧거나 일반적	prompt에 컨텍스트 부족	관련 파일과 목표 명시
권한 오류	tools 누락 또는 deny 적용	tools 필드 추가, 정책 검토
비용 폭등	Opus 사용 또는 prompt 비대	모델 변경, 입력 축소
결과 형식 깨짐	시스템 프롬프트 출력 형식 모호	예시 포함하여 형식 강제

/agents

등록 목록 확인

/cost

비용 점검

--debug

트레이스 모드

Logs

~/claude/logs

실전 / 자동 디스패치

대화 예시

auto dispatch demo

> 어제 머지된 PR #234의 보안과 테스트를 한 번에 점검해 줘.

[메인] 보안과 테스트를 동시에 점검하겠습니다.
2개의 Subagent를 병렬로 호출합니다.

→ Task(security-scanner, "PR #234 diff and dependencies")
→ Task(test-writer, "PR #234 changed functions coverage")

[보안] 분석 중... 의존성 3개에 알려진 CVE 발견. 자세히 확인 중.

[테스트] 분석 중... 변경된 5개 함수 중 2개가 테스트 누락.

[메인] 결과 종합:
보안: 의존성 3건 (CVE-2024-1234 등). 패치 권장.
테스트: 누락 2건. 추가 작성 가능. 진행하시겠어요?

> 네, 진행해 주세요.

2 agents

병렬 호출

Auto

명시 없이 선택

Aggregate

결과 집계

Next-step

다음 단계 제안

실전 / 명시적 호출

특정 에이전트 강제 지정

explicit demo

```
> /agents
Available agents:
- code-reviewer : PR 코드 리뷰 (Sonnet)
- security-scanner : 보안 분석 (Opus)
- test-writer : 테스트 작성 (Sonnet)
- docs-writer : 문서 작성 (Sonnet)
- migration-bot : 일괄 변경 (Haiku)

> docs-writer를 사용해서 src/api/users.js의 API 문서를 작성해 줘.

[메인] docs-writer 에이전트를 호출합니다.
→ Task(docs-writer, "Generate API doc for src/api/users.js")

[docs ] src/api/users.js 분석 중...
5개 엔드포인트 식별: GET, POST, PUT, DELETE, PATCH /users
OpenAPI 스타일 마크다운 작성 중...
docs/api/users.md 생성 완료.

[메인] 문서 작성 완료: docs/api/users.md (5 endpoints documented).
미리보기를 원하시면 보여드릴 수 있습니다.
```

/agents

목록 확인

By name

명시 호출

Predictable

예측 가능

Preview

확인 요청

Task 도구 Best Practices

실무에서 검증된 가이드

TIP 1

명확한 prompt

목표, 관련 파일, 출력 형식을 구체적으로 명시.
짧을수록 위험.

TIP 2

병렬화 활용

서로 독립적 작업은 한 번에 모두 호출.
시간을 크게 단축.

TIP 3

모델 선택

Subagent 정의 시 작업 복잡도에 맞는 모델을 명시 지정.

TIP 4

결과 형식 강제

JSON 또는 마크다운 섹션 같은 구조화 출력을 시스템 프롬프트에서 지시.

ANTI-PATTERN

한 Subagent에 너무 많은 책임을 주지 마세요. 큰 작업은 여러 전문 Subagent로 분할해 병렬 호출하는 것이 시간과 품질 모두에 유리.

Specific

구체적 지시

Parallel

병렬 활용

Right model

모델 적합

Structured

구조화 출력

Part 3 Recap

Task 도구 핵심 정리

TAKEAWAY 1

Task = 유일한 진입점

Subagent 호출은 모두
Task 도구를 통해.
subagent_type/description/prompt 3요소

TAKEAWAY 2

두 가지 디스패치

자동(description 매칭)과
명시(이름 지정), 명시가 우선

TAKEAWAY 3

병렬 우선

독립 작업은 병렬로, 의존 작업은 직렬로. 시간 절감을 적극 추구

TAKEAWAY 4

관찰 가능성

/cost, /agents, --debug
로 비용과 동작을 항상 추적

NEXT - PART 4

Part 4부터는 5가지 실전 패턴을 차례로 다룹니다.

Part 4는 Pattern 1 Code Reviewer로 PR 자동 리뷰 에이전트를 12장에 걸쳐 구현합니다.

Part 3

완료

Part 4

Code Reviewer (12p)

Part 5-8

4 patterns (38p)

~2.5h

잔여 시간

CHAPTER 02 – PART 04

Agents – Code Reviewer

Subagents 시스템과 실전 패턴



Part 4 / Pattern 1 - Code Reviewer

PR 자동 리뷰 에이전트 완전 구현

PATTERN OVERVIEW

Code Reviewer 패턴은 가장 활용도가 높은 Subagent 패턴. PR diff를 자동으로 분석해 개선 사항을 제안하고 CI/CD에도 통합 가능.

STEP 1

시나리오 설계

리뷰 대상, 검토 관점, 출력
형식, 사용 워크플로를 정의.

STEP 2

에이전트 정의

frontmatter와 시스템 프
롬프트를 작성하고 권한을
좁힘.

STEP 3

호출 패턴

대화형, 헤드리스, GitHub
Actions 세 가지 호출 방법
을 다룸.

STEP 4

확장 변형

언어별 특화, 보안 통합, 멀티
에이전트 협업 같은 변형을
살펴 봄.

12 Slides

Part 4 분량

~25 min

예상 강의 시간

Production

실무 수준 예시

CI-ready

CI 통합 가능

시나리오 정의

Code Review가 해결할 문제

WHO

대상 사용자

개인 개발자 또는 팀 전체. 본인 PR을 머지 전 셀프 리뷰하거나 동료 PR을 분석.

WHEN

사용 시점

PR 작성 직후 자동 호출 또는 사용자가 직접 검토 요청 시.

WHY

해결하는 문제

리뷰 응답 지연, 일관성 부족, 단순 이슈에 시니어 시간 낭비를 줄임.

BOUNDARY

이 에이전트는 분석과 제안만 함. 자동 머지나 직접 수정은 하지 않음. 사람의 최종 판단을 존중.

Pre-merge

머지 전 리뷰

Consistent

일관된 기준

Fast

즉시 응답

Suggest

제안만 함

에이전트 정의 완성본

.claude/agents/code-reviewer.md

code-reviewer.md

```
---
name: code-reviewer
description: !
  PR diff와 변경된 파일을 검토하여 가독성, 안전성, 성능,
  테스트 커버리지 관점에서 구체적인 개선 사항을 제안합니다.
  사용 시절: 코드 리뷰가 필요한 경우, git diff 분석이 필요한 경우.
tools: Read, Grep, Glob, Bash(git diff:*), Bash(git log:*), Bash(git show:*)
model: claude-sonnet-4-5
---
```

당신은 10년 경력의 시니어 소프트웨어 엔지니어입니다.

검토 우선순위

1. 안전성 critical: null 참조, race condition, 입력 검증 누락
2. 가독성 high: 명명, 함수 크기, 주석 필요성
3. 성능 medium: 알고리즘 복잡도, 불필요한 I/O
4. 테스트 medium: 새 코드의 테스트 커버리지

출력 형식 (마크다운)

Summary

한 단락 요약

Issues (우선순위순)

- [severity] file.js:42 - 문제 설명. 권장 수정안.

Sonnet

균형 모델

Read-only

안전 권한

4 priorities

검토 우선순위

Markdown

구조화 출력

시스템 프롬프트 심화

리뷰 품질을 결정하는 디테일

prompt details

추가 권장 섹션

행동 규칙

- 사실에 기반하여 비판: 추측이나 의견 단정 금지
- 코드 컨텍스트 확인: 호출 지점, 데이터 흐름까지 추적
- 트레이드오프 인정: 절대적 판단보다 옵션 제시
- 사소한 스타일은 자동 포맷터 권장으로 대체

무시할 사항

- 작성자의 개인 스타일 (자동 포맷에 위임)
- 컨벤션 외 미적 취향
- 추측에 기반한 잠재 문제

좋은 코멘트 예시

- "auth.js:42 - req.user가 nil일 수 있음 (line 38의 미들웨어 실패 시).
가드 추가 권장: if (!req.user) return res.status(401)..."

나쁜 코멘트 예시

- "이 함수가 좀 이상해 보임" (구체성 부족)
- "이렇게 하면 더 깔끔" (취향 강요)

Fact-based

사실 기반

Specific

구체적

Actionable

실행 가능

Respectful

존중하는 톤

호출 방법 1 / 대화형

일상적인 PR 리뷰 흐름

interactive

```
# 본인 PR을 작성한 후
$ git push origin feat/new-payment-flow
$ gh pr create --title "신규 결제 흐름" --body "..."
```

```
# Claude Code 세션에서 셀프 리뷰
$ claude
```

> 방금 만든 PR을 머지 전에 셀프 리뷰해 주세요.

```
[메인] code-reviewer를 호출해 분석합니다.
      → Task(code-reviewer, "git diff main...HEAD")
```

[리뷰] 분석 중... 변경 파일 7개, +234 -56 line.

```
## Summary
신규 결제 흐름 추가. 핵심 변경 사항은 PaymentService와 idempotency key.
```

```
## Issues
- [high] payment.js:88 - refund 흐름에서 idempotency key가 누락
- [medium] payment.js:124 - DB 트랜잭션 경계가 모호
- [low] tests/payment.test.js - refund 케이스 누락
```

> 첫 번째 이슈를 자세히 설명해 줘.

Self-review

머지 전

Conversation

대화형

Drill-down

상세 파고들기

Iterative

반복 개선

호출 방법 2 / 헤드리스

스크립트와 자동화 친화

headless

```
# 헤드리스 모드로 단일 호출
$ git diff main..HEAD | claude -p \
  "code-reviewer 에이전트로 이 PR diff를 검토해 주세요" \
  --output-format json \
  > review-result.json

# JSON 구조
{
  "summary": "신규 결제 흐름 추가...",
  "issues": [
    { "severity": "high", "file": "payment.js", "line": 88,
      "message": "...", "suggestion": "..." },
    { "severity": "medium", "file": "payment.js", "line": 124, ... }
  ],
  "tokens_used": 8234,
  "cost_usd": 0.041
}

# jq로 후처리
$ jq '.issues | map(select(.severity == "high"))' review-result.json

# pre-commit hook으로 활용 가능
```

-p

헤드리스 플래그

JSON

구조화 출력

jq

필드 추출

hook

자동화 통합

호출 방법 3 / GitHub Actions

PR 생성 시 자동 댓글 게시

GitHub Actions

```
# .github/workflows/claude-review.yml
name: Claude Code Review
on:
  pull_request:
    branches: [main]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }
      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code
      - name: Run Code Reviewer
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          git diff origin/main...HEAD | claude -p \
            "code-reviewer로 이 PR을 검토" \
            --allowed-tools "Read,Grep,Glob" \
            --output-format text > review.md
      - name: Post Review Comment
        uses: actions/github-script@v7
        with:
          script: |
            const review = require('fs').readFileSync('review.md', 'utf8');
            github.rest.issues.createComment({ ... body: review ... });
```

on PR
트리거

API key
secrets 활용

allowed-tools
CI에서 권한 줌

PR comment
결과 자동 게시

확장 변형

Code Reviewer의 다양한 버전

VARIANT 1

언어별 특화

js-reviewer, py-reviewer, go-reviewer
등 언어별 관용구와 베스트
프랙티스 강화

VARIANT 2

도메인 특화

frontend-reviewer
(a11y, perf), backend-reviewer (보안, DB), ml-reviewer (재현성)

VARIANT 3

체크리스트 기반

팀의 PR 체크리스트를 시스템
프롬프트로 변환하여 일
관성 강화

VARIANT 4

학습 모드

시니어 리뷰 결과를 모아
examples로 시스템 프롬프
트에 포함, 팀 스타일 학습

COMPOSITION

여러 변형을 동시에 운영하면 메인 에이전트가 description 매칭으로 적절한 변형을 자동 선택.

예를 들어 .py 파일이면 py-reviewer가 자동 호출.

Language

언어 특화

Domain

도메인 특화

Checklist

체크리스트

Learning

예시 학습

멀티 에이전트 협업

Code Reviewer와 다른 에이전트의 조합

multi-agent

시나리오: PR 종합 점검 (4개 에이전트 병렬)

> PR #234를 머지 전에 전반적으로 점검해 주세요.

[메인] 4개 에이전트를 병렬 호출합니다.

- Task(code-reviewer) 코드 품질
- Task(security-scanner) 보안 이슈
- Task(test-coverage) 테스트 부족
- Task(docs-checker) 문서 동기화 여부

결과 종합 (메인이 자동 집계)

종합 보고서

Critical (5건)

- [security] auth.js:42 ...
- [code] payment.js:88 ...
- ...

Recommended (8건)

- [test] ...
- [docs] ...

머지 가능 여부: Critical 해결 후 가능

사용자는 단일 명령으로 4관점 보고서 획득

4 agents

동시 호출

Parallel

병렬 실행

Aggregate

메인이 집계

Verdict

머지 가능 여부

성능 튜닝

리뷰 응답 시간과 비용 최적화

Goal	Tactic	효과
응답 시간 단축	Sonnet → Haiku 일부 작업 전환	50% 시간 절감
비용 절감	tools를 Read/Grep만으로 좁힘	Bash 호출 줄이며 토큰 감소
품질 향상	주제별 시스템 프롬프트 examples 추가	구체성/일관성 향상
컨텍스트 절약	diff를 prompt에 압축해 전달	입력 토큰 30% 절감
병렬화 효율	Subagent 수를 4 이하로 제한	오버헤드 누적 방지

MEASURE FIRST

튜닝 전후 /cost와 응답 시간을 반드시 비교 측정하세요. 직관과 다른 결과가 자주 나옵니다.

Measure

측정 우선

Iterate

반복 개선

Trade-off

품질 vs 비용

Documented

결정 기록

자주 만나는 함정

실무 도입 시 주의 사항

PITFALL 1

과도한 코멘트

에이전트가 사소한 이슈까지 모두 지적하면 신호가 묻힙니다.
우선순위로 압축.

PITFALL 2

컨텍스트 부재 분석

관련 호출 지점을 모르고 단일 함수만 보면 오판이 잦습니다.
Grep 활용 유도.

PITFALL 3

취향 강요

팀 컨벤션 외 영역에서 취향을 강요하면 거부감 발생.
시스템 프롬프트에 명시.

PITFALL 4

신뢰 형성 지연

도입 초기에 오탐이 많으면 사용자가 외면.
초기엔 high severity만 보고.

ADOPTION ROADMAP

초기에는 셀프 리뷰 보조 도구로 도입하고 한 달간 정확도를 검증한 뒤 CI에 통합하세요.
처음부터 모든 PR에 자동 적용은 신뢰 형성에 실패할 수 있습니다.

Prioritize

우선순위 압축

Context

맥락 추적

Respect

취향 존중

Trust

점진 신뢰

Part 4 Recap

Code Reviewer 핵심 정리

TAKEAWAY 1

완성된 정의

frontmatter 4필드와 우선
순위 명시 시스템 프롬프트
로 즉시 사용 가능

TAKEAWAY 2

3가지 호출

대화형, 헤드리스, GitHub
Actions를 상황에 맞게 선택

TAKEAWAY 3

4가지 변형

언어, 도메인, 체크리스트,
학습 모드로 확장 가능

TAKEAWAY 4

점진 도입

셀프 리뷰부터 시작해 신뢰
형성 후 CI 통합으로 확장

NEXT - PART 5

Part 5는 Pattern 2 Tester로 테스트 커버리지를 자동 확장하는 에이전트를 10장에 걸쳐 구현합니다.

Part 4

완료

Part 5

Tester (10p)

Part 6-8

Security/Docs/Migration

~2h

잔여 시간

CHAPTER 02 – PART 05

Agents – Tester

Subagents 시스템과 실전 패턴



Part 5 / Pattern 2 - Tester

테스트 커버리지 자동 확장 에이전트

PATTERN OVERVIEW

Tester 패턴은 코드 커버리지를 분석하고 누락된 엣지 케이스, 분기, 예외 경로에 대한 테스트를 자동 작성.

INPUT

입력

대상 함수 또는 파일, 기존
테스트 파일, 커버리지 리포
트(있다면)

OUTPUT

출력

새 테스트 파일, 추가 테스트
케이스, 커버리지 변화 보고
서

MODEL

모델 선택

Sonnet 기본. 복잡한
mock 설계가 필요하면
Opus 선택 가능

TOOLS

필요 도구

Read, Edit, Write,
Bash(npm test 또는
pytest), Grep

10 Slides

Part 5 분량

~20 min

예상 강의 시간

Write tests

직접 작성

Verify

실행 검증

test-writer 정의

.claude/agents/test-writer.md

test-writer.md

```
---
name: test-writer
description: |
  대상 함수나 파일의 테스트 커버리지를 분석해 누락된 엡지 케이스, 분기,
  예외 경로에 대한 단위 테스트를 작성. 작성 후 실제로 실행하여 통과 확인.
  사용 시점: 새 함수에 테스트가 필요하거나 커버리지 부족이 확인된 경우.
tools: Read, Grep, Glob, Edit, Write, Bash(npm test:*), Bash(pytest:*), Bash(jest:*)
model: claude-sonnet-4-5
---
```

당신은 테스트 자동화 전문가입니다.

작업 절차

1. 대상 함수의 시그니처와 분기 구조 분석
2. 기존 테스트 파일이 있다면 스타일 일치
3. 정상, 엡지, 예외 케이스 모두 커버
4. 테스트 작성 후 실행하여 통과 확인
5. 통과한 케이스만 최종 결과로 반환

작성 원칙

- 한 테스트가 한 가지만 검증
- 의미 있는 테스트 이름 (describe.it 또는 test_name)
- AAA 패턴 (Arrange Act Assert)
- 외부 의존성은 mock 적절히 사용

Sonnet

균형 모델

Write+Exec

작성 + 실행

AAA

테스트 패턴

Mock

의존성 처리

커버리지 확장 워크플로

7단계 자동 흐름

STEP 01

커버리지
측정

STEP 02

갭 식별
(파일/함수)

STEP 03

대상 코드
분석

STEP 04

테스트
작성

Subagent의 실제 진행 흐름

- | | |
|---------------------------------------|-------------|
| 1. Bash: npm test -- --coverage | → 현재 67% 측정 |
| 2. Read: coverage/lcov-report/summary | → 갭 위치 식별 |
| 3. Grep: 미커버 함수의 호출 지점 분석 | → 입력 패턴 파악 |
| 4. Read: 대상 함수 본문 | → 분기와 조건 분석 |
| 5. Read: 기존 테스트 (있다면) | → 스타일 학습 |
| 6. Write: tests/new.test.js | → 신규 테스트 작성 |
| 7. Bash: npm test -- new.test.js | → 실행 통과 확인 |
| → 실패 시 자동 수정 후 재실행 (최대 2회) | |
| 8. Bash: npm test -- --coverage | → 92% 달성 확인 |
| 9. Return: 변경 요약 + 커버리지 향상 보고 | |

workflow

67% → 92%

예시 향상

Auto-fix

실패 자동 수정

Verify

실행 검증

Report

결과 보고

엣지 케이스 자동 발굴

단순 라인 커버리지를 넘어서

Category	에이전트가 자동 발굴하는 케이스	예시
Boundary	경계값 (0, 1, max, max+1)	empty array, single item
Null/Undefined	값 없는 경우의 처리	optional fields
Exception	예외 발생 경로	network failure, timeout
Concurrency	동시성 충돌 시나리오	race condition
Invariant	함수 호출 전후 불변 조건	idempotency, total balance

BEYOND LINE COVERAGE

Tester 에이전트는 단순 라인 커버리지가 아닌 분기 커버리지와 의미 있는 케이스에 집중.

100% 라인 커버리지보다 핵심 분기의 90% 커버가 더 가치 있습니다.

5 categories

엣지 카테고리

Branch

분기 커버리지

Meaningful

의미 있는 테스트

Quality

품질 우선

실전 대화 예시

PaymentService 커버리지 확장

> PaymentService의 테스트 커버리지를 90%로 끌어올려 주세요.

real dialog

[메인] test-writer 에이전트를 호출합니다.
→ Task(test-writer, "PaymentService coverage to 90%")

[test] 현재 커버리지 측정 중...
Bash: npm test -- --coverage
결과: PaymentService 67%
캡 분석 중...
Read: coverage/lcov-report/payment.html
미커버: refund(), retry(), idempotency_check()

refund() 분석 중...
Read: src/services/payment.js (line 88-145)
분기 6개, 엣지 케이스 4개 식별

테스트 작성 중...
Write: tests/payment.refund.test.js (8 cases)

실행 검증 중...
Bash: npm test -- payment.refund
결과: 8/8 passed ✓

최종 커버리지 측정...
결과: PaymentService 92% (+25%)

[메인] 92%까지 향상. 8개 케이스가 신규 추가됨.

67% → 92%

향상폭

8 cases

신규 케이스

8/8 passed

모두 통과

~3 min

실행 시간

테스트 프레임워크 지원

주요 언어와 도구별 적응

Language	Framework	명령
JavaScript/TS	Jest, Vitest, Mocha	npm test 또는 npx vitest
Python	pytest, unittest	pytest -v
Go	testing 표준 패키지	go test ./...
Java	JUnit 5, TestNG	mvn test 또는 gradle test
Rust	cargo test (표준)	cargo test
Ruby	RSpec, Minitest	rspec 또는 rake test

Auto-detect	6+ langs	Convention	CI-ready
자동 감지	주요 언어	관례 존중	CI 친화

Mock 전략

외부 의존성 처리 방법

NETWORK

네트워크 호출

HTTP 클라이언트를 mock으로 대체.
응답 시나리오 명시.

DATABASE

데이터베이스

In-memory DB 또는 ORM mock.
쿼리 결과를 fixture로 준비.

TIME

시간 의존

Date.now 또는 time.time을 freeze.
시간 관련 분기 테스트.

FILESYSTEM

파일 시스템

memfs 또는 tmp 디렉토리.
실제 파일 생성 회피.

PHILOSOPHY

Mock은 필요한 만큼만 사용. 과도한 mock은 실제 동작과의 괴리를 만들며, Integration test는 별도로 운영하는 것을 권장.

HTTP

API mock

DB

in-memory

Time

freeze

FS

memfs/tmp

테스트 품질 지표

에이전트가 자가 평가하는 기준

COVERAGE

커버리지

Line, Branch, Function 모두 확인.
Branch 80% 이상 권장.

CASES

케이스 다양성

정상, 경계, 예외, 동시성 4종 모두 포함
되었는지 자체 점검.

EXEC

실행 성공

모든 작성 테스트가 실제로 통과하는지
확인. 실패 시 자동 수정.

SELF-EVAL

에이전트는 작업 결과를 메인에 반환할 때 위 3가지 지표를 자가 평가하여 보고. 사용자는 보고만으로 품질을 빠르게 파악할 수 있음.

Coverage

범위 측정

Diversity

케이스 다양성

Verify

실행 검증

Self-eval

자가 평가

CI 통합

PR마다 자동 커버리지 보강

CI workflow

```
# .github/workflows/auto-test.yml
name: Auto Test Coverage
on:
  pull_request:
    paths: ['src/**']

jobs:
  extend-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci && npm test -- --coverage

      - name: Extend coverage if low
        if: failure() # 커버리지 한계 미달 시
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "test-writer로 PR 변경 함수의 커버리지를 90% 이상으로" \
            --allowed-tools "Read,Edit,Write,Bash(npm test:*)"

      - name: Open PR for new tests
        run: gh pr create --title "auto-tests for #${{ github.event.number }}"
```

On PR
트리거

Conditional
미달 시만

Auto PR
신규 PR 자동 생성

Closed loop
닫힌 루프

Part 5 Recap

Tester 핵심 정리

TAKEAWAY 1

의미 있는 테스트

Line 커버리지보다 Branch
와 엡지 케이스에 집중

TAKEAWAY 2

실행 검증

작성 후 반드시 실제 실행해
통과 확인

TAKEAWAY 3

Mock 균형

필요한 만큼만 mock 사용,
과도한 mock 회피

TAKEAWAY 4

CI 자동화

커버리지 미달 시 별도 PR로
자동 보강 가능

NEXT - PART 6

Part 6은 Pattern 3 Security Scanner입니다. 취약점과 의존성, 시크릿을 자동 점검하는 보안 전문 에이전트를 10장에 걸쳐 구현합니다.

Part 5

완료

Part 6

Security (10p)

Part 7

Docs (10p)

Part 8

Migration (8p)

CHAPTER 02 – PART 06

Agents – Security Scanner

Subagents 시스템과 실전 패턴



Part 6 / Pattern 3 - Security Scanner

보안 점검 전문 에이전트

PATTERN OVERVIEW

코드, 의존성, 설정 파일을 보안 관점에서 점검하는 에이전트. 시크릿 유출, 취약점, 권한 오용 같은 위협을 식별합니다.

CHECK 1

코드 취약점

SQL injection, XSS,
CSRF, 인증 우회, 권한 우회
같은 OWASP Top 10

CHECK 2

의존성

package.json,
requirements.txt 등의 알
려진 CVE 검출

CHECK 3

시크릿

API key, 토큰, 비밀번호의
하드코딩 발견

CHECK 4

설정

CORS, CSP, 권한 설정의
부적절한 구성 점검

10 Slides

Part 6 분량

Opus

심층 분석 모델

4 categories

점검 영역

Read-only

안전 권한

security-scanner 정의

OWASP Top 10 기반 점검

```
---
name: security-scanner
description: |
  코드, 의존성, 설정 파일에서 보안 취약점을 식별. OWASP Top 10 기반으로
  injection, XSS, 권한 오용, 시크릿 노출, 의존성 CVE를 분석.
  사용 시점: 보안 점검이 명시적으로 요청되거나 새 PR의 보안 검토 시.
tools: Read, Grep, Glob, Bash(npm audit), Bash(pip check), Bash(safety check)
model: claude-opus-4-5
---
```

security-scanner.md

당신은 보안 엔지니어입니다.

점검 카테고리 (OWASP 기반)

1. Injection (SQL, NoSQL, OS command, LDAP)
2. Broken Authentication
3. Sensitive Data Exposure (시크릿, PII)
4. XML External Entities (XXE)
5. Broken Access Control
6. Security Misconfiguration
7. Cross-Site Scripting (XSS)
8. Insecure Deserialization
9. Known Vulnerable Components
10. Insufficient Logging & Monitoring

출력 형식 (필수)

- Severity: critical | high | medium | low | info
- CWE 번호와 OWASP 카테고리
- 파일:라인, 재현 시나리오, 수정 방법

Opus

심층 모델

OWASP 10

점검 기준

CWE

취약점 번호

Severity

심각도 분류

취약점 탐지 능력

코드와 의존성 양면 점검

Vulnerability	탐지 방법	예시	
SQL Injection	문자열 연결 쿼리 패턴 + 외부 입력 추적	`query("SELECT * WHERE id="+id)`	
XSS	innerHTML, dangerouslySetInnerHTML 사용 분석	React dangerouslySetInnerHTML	
Auth Bypass	미들웨어 우회 경로, 권한 체크 누락 탐색	관리자 라우트의 미들웨어 누락	
Secret Leak	정규식으로 키 패턴 검사 + 커밋 이력 점검	AKIA... AWS 키 하드코딩	
CVE Match	npm/pip audit + 매니페스트 분석	lodash 4.17.20 CVE-2021-23337	
Static 정적 분석	Audit 의존성 도구	Pattern 패턴 매칭	Context 데이터 흐름

실전 대화 예시

PR 보안 점검

security demo

> 방금 머지된 PR #245의 보안 점검을 해 주세요.

[메인] security-scanner를 호출합니다.
→ Task(security-scanner, "PR #245 보안 점검")

[sec] PR diff 분석 중...
Bash: git diff main..245 → 12 files changed

의존성 점검 중...
Bash: npm audit → 3 vulnerabilities found

코드 분석 중...
Read+Grep: SQL 패턴, eval, exec, innerHTML

시크릿 검사 중...
Grep: API_KEY|TOKEN|PASSWORD patterns

Security Report

Critical (2)

- [SQL Injection] src/api/users.js:78 (CWE-89)
외부 input이 query 문자열에 직접 삽입.
수정안: parameterized query 사용
- [Secret Leak] src/config/aws.js:12 (CWE-798)
AKIA로 시작하는 AWS 키 하드코딩.
수정안: 환경변수로 이동

High (1)

- [Outdated Dep] lodash 4.17.19 → 4.17.21
CVE-2021-23337 (Prototype Pollution)

[메인] Critical 2건, High 1건 발견. 머지 차단 권장.

12 files

변경 파일 수

2+1 issues

발견 이슈

Critical

최고 심각도

Block merge

머지 차단 권장

의존성 스캐닝

npm audit / pip-audit 통합

dependency scan

```
# 에이전트가 사용하는 표준 도구들

# JavaScript/TypeScript
npm audit --json
yarn audit --json
pnpm audit --json

# Python
pip-audit --format json
safety check --json

# Ruby
bundle audit check --update

# Go
govulncheck ./...

# 에이전트가 결과를 통합 분석
- 같은 CVE라도 컨텍스트별 영향도 평가
- 직접 사용 vs 간접 의존성 구분
- 패치 경로 제안 (직접 업데이트 vs 우회)
- 패치 불가 시 mitigation 제안
```

Multi-tool

도구 통합

Context

영향도 평가

Direct/Tran

의존성 구분

Mitigation

우회책 제안

시크릿 탐지

하드코딩과 커밋 이력 점검

secret patterns

에이전트가 검색하는 시크릿 패턴

AWS

AKIA[0-9A-Z]{16}

Access Key

aws_secret_access_key\s*=\s*['"]

Secret Key

Google Cloud

AIza[0-9A-Za-z-_]{35}

API Key

service_account[\s\S]*?private_key

JSON SA

GitHub

ghp_[0-9a-zA-Z]{36}

Personal Token

gho_[0-9a-zA-Z]{36}

OAuth Token

Generic

api[~_]?key\s*=\s*['"]{20,}

일반 API key

password\s*=\s*['"]{1,}+['"]

password

추가 점검

- .env 파일 git 추적 여부

- 과거 커밋의 시크릿 (git log -p)

- .gitignore 누락 위험

Pattern

정규식 매칭

Multi-vendor

클라우드별

History

커밋 이력

.env

파일 점검

Pre-commit Hook 통합

커밋 직전 자동 차단

pre-commit

```
# .git/hooks/pre-commit (또는 husky 등 사용)
#!/bin/bash

# 1. 스테이지된 파일에서 시크릿 검사
git diff --cached | claude -p \
  "security-scanner로 이 변경에 시크릿이 있는지 검사" \
  --output-format json > /tmp/sec-check.json

# 2. critical 시 커밋 차단
CRITICAL=$(jq '.issues[] | select(.severity == "critical") | length' \
  /tmp/sec-check.json)

if [ "$CRITICAL" -gt 0 ]; then
  echo "❌ Critical security issues found. Commit blocked."
  jq '.issues[] | select(.severity == "critical")' /tmp/sec-check.json
  exit 1
fi

echo "✓ Security check passed."
exit 0

# 사용자 경험
$ git commit -m "feat: new feature"
[claude 실행 중...]
❌ Critical security issues found. Commit blocked.
[Secret Leak] src/config.js:5 - AWS_SECRET_ACCESS_KEY 하드코딩
```

Hook
Git pre-commit

Block
커밋 차단

Fast
빠른 검사

Severity
critical 만 차단

오탐 처리

False Positive 줄이기

STRATEGY 1

컨텍스트 추적

외부 입력이 실제로 위험 함수에 도달하는지 데이터 흐름 분석

STRATEGY 2

주석 기반 제외

```
// security-scanner:  
ignore 주석으로 명시적 제외 가능
```

STRATEGY 3

베이스라인

기존 이슈를 baseline.json에 저장해 신규 이슈만 보고

STRATEGY 4

환경별 규칙

dev/staging/prod 환경별로 검사 강도 조정

TRUST BUILDING

도입 초기에 오탐이 많으면 사용자가 외면합니다. baseline을 활용해 신규 이슈만 보여주는 점진 도입이 효과적입니다.

Data-flow

흐름 추적

Annotation

주석 제외

Baseline

기존 분리

Per-env

환경별

컴플라이언스 통합

SOC2, GDPR 같은 표준 매핑

Compliance	관련 점검 항목	에이전트 매핑
SOC2	접근 통제, 변경 관리, 암호화	권한 체크, secret 검사
GDPR	개인정보 처리, 동의, 삭제	PII 노출 검사
PCI DSS	카드 데이터 보호, 키 관리	시크릿/암호화 검사
HIPAA	의료 정보 보호, 감사 로그	PHI 노출, 로깅 점검
ISO 27001	정보보안 관리 체계 전반	종합 보안 보고서

CUSTOM RULES

조직 고유의 보안 정책은 시스템 프롬프트에 추가하여 에이전트가 함께 점검하도록 할 수 있음.

SOC2

미국 표준

GDPR

EU 개인정보

PCI

카드 결제

Custom

조직 정책

Part 6 Recap

Security Scanner 핵심 정리

TAKEAWAY 1

4가지 점검

코드 취약점, 의존성 CVE, 시
크릿, 설정 부적절

TAKEAWAY 2

OWASP 기반

Top 10 카테고리화 CWE 번
호로 표준 분류

TAKEAWAY 3

Opus 권장

심층 분석이 필요한 보안 영역
, 비용 정당화 가능

TAKEAWAY 4

점진 도입

baseline으로 기존 이슈 분리
, 신규만 차단으로 신뢰 형성

NEXT - PART 7

Part 7은 Pattern 4 Docs Writer입니다. README, API 문서, ADR을 자동 작성하는 문서 에이전트를 10장에 걸쳐 구현합니다.

Part 6

완료

Part 7

Docs Writer (10p)

Part 8

Migration (8p)

Part 9

Recap (10p)

CHAPTER 02 – PART 07

Agents – Docs Writer

Subagents 시스템과 실전 패턴



Part 7 / Pattern 4 - Docs Writer

문서 자동 생성 에이전트 구현

PATTERN OVERVIEW

Docs Writer는 README, API 문서, ADR 같은 기술 문서를 코드와 일치하도록 작성 또는 업데이트하는 에이전트입니다.

TARGETS

대상 문서

README, API 레퍼런스,
ADR, 변경 로그, 사용자 가
이드를 다룹니다.

PRINCIPLE

핵심 원칙

코드가 진실의 원천이고 문
서는 그에 동기화되어야 합
니다.

MODEL

모델 선택

Sonnet이 기본, 대규모 다
국어 문서는 Opus 활용 가
능합니다.

TRIGGER

실행 시점

신규 기능 추가, API 시그니
처 변경, ADR 결정 시 자동
호출됩니다.

10 Slides

Part 7 분량

~20 min

예상 강의 시간

4 types

문서 카테고리

Sync

코드 동기화

docs-writer 에이전트 정의

완성된 정의 파일

docs-writer.md

```
---
name: docs-writer
description: |
  코드 변경에 맞춰 README, API 문서, ADR을 작성 또는 업데이트.
  사용 시점: 신규 기능 추가, API 시그니처 변경, 아키텍처 결정이
  있을 때.
tools: Read, Grep, Glob, Edit, Write, Bash(git log:*)
model: claude-sonnet-4-5
---
```

당신은 기술 문서 작성가입니다.

작성 절차

1. 대상 코드를 Read로 읽어 실제 동작 확인
2. 기존 문서가 있으면 토과 형식 유지
3. 변경 사항을 명확하게 반영
4. 예시는 실제 실행 가능한 것으로 작성
5. 한국어와 영어 혼용 시 일관성 유지

문서 종류별 가이드

- README: 개요 → 설치 → 사용법 → 예시
- API: 엔드포인트 → 파라미터 → 응답 → 예시
- ADR: Context → Decision → Consequences

Sonnet

균형 모델

Edit/Write

문서 직접 작성

3 types

README/API/ADR

Bilingual

한영 혼용 가능

README 자동 생성

신규 프로젝트 README 작성 예시

README example

```
# 사용자 요청
"이 프로젝트의 README를 작성해 줘"

# docs-writer의 작업 흐름
1. Read package.json → 프로젝트명, 의존성, 스크립트 파악
2. Read src/index.js → 진입점과 주요 함수 파악
3. Glob examples/**/*.js → 예시 코드 수집
4. Bash git log --oneline -10 → 최근 변경 동향
5. Write README.md (다음 구조로):

# Project Name
한 문장 설명.

## Installation
```
npm install
```

## Usage
```js
const lib = require('./src');
lib.start({ port: 3000 });
```

## Examples
examples/ 디렉토리 참고.

## License
MIT
```

Auto-discover

구조 자동 탐색

Examples

예시 코드 수집

Structured

표준 섹션 구성

Runnable

실행 가능 예시

API 문서 생성

엔드포인트 자동 문서화

API doc

```
# 사용자 요청
"src/routes/users.js의 엔드포인트 문서를 만들어 줘"
```

```
# docs-writer의 출력 (Markdown)
## GET /users
```

사용자 목록을 페이지네이션과 함께 반환합니다.

```
### Query Parameters
```

| Name | Type | Required | Default | Description |
|-------|------|----------|---------|----------------------|
| page | int | No | 1 | 페이지 번호 (1-base) |
| limit | int | No | 20 | 페이지당 항목 수 (max: 100) |

```
### Response 200
```

```
``json
{
  "users": [{ "id": "u_1", "name": "..."}],
  "links": { "next": "/users?page=2", "prev": null },
  "total": 142
},
```

```
### Errors
```

```
- 400: limit > 100
```

Routes

엔드포인트 분석

Params

파라미터 추출

Schema

응답 스키마

Errors

에러 코드

ADR 작성

Architecture Decision Record

ADR example

```
# 사용자 요청
"PostgreSQL을 MongoDB로 교체한 결정에 대한 ADR을 작성해 줘"
```

```
# docs-writer 출력 (docs/adr/0008-mongodb-migration.md)
# ADR 0008: PostgreSQL에서 MongoDB로 전환
```

```
Status: Accepted
Date: 2026-05-25
Deciders: 백엔드 팀
```

```
## Context
- 사용자 프로필 데이터가 빠르게 진화하는 스키마를 가짐
- PostgreSQL의 alter table 대기 시간이 운영에 영향
- 읽기 부하가 쓰기의 100배 (스케일 아웃 필요)
```

```
## Decision
사용자 프로필 도메인만 MongoDB Atlas로 전환.
주문/결제 도메인은 PostgreSQL 유지.
```

```
## Consequences
+ 스키마 변경이 마이그레이션 없이 가능
+ 읽기 스케일이 사당으로 확장
- 트랜잭션 보장 약화 (보상 트랜잭션 패턴 도입)
- 운영팀이 두 DB 모두 관리
```

Status

결정 상태

Context

배경 설명

Decision

결정 내용

Consequences

결과 영향

다국어 문서

한국어와 영어 동시 관리

i18n docs

패턴 1: 두 디렉토리 분리

```
docs/  
├── en/  
│   ├── README.md  
│   └── api.md  
└── ko/  
    ├── README.md  
    └── api.md
```

패턴 2: 한 파일에 섹션 분리 (작은 프로젝트)

README.md

English

This project provides a payment gateway integration.

한국어

이 프로젝트는 결제 게이트웨이 통합을 제공합니다.

docs-writer 호출

```
> @docs-writer src/payment.js 변경 사항을  
docs/en/payment.md와 docs/ko/payment.md에 모두 반영해 줘
```

결과: 두 언어 버전이 동시에 갱신됨

en/ko

디렉토리 분리

Single file

섹션 분리

Sync

동시 갱신

Consistency

용어 일관성

Changelog 자동 생성

git log 기반 변경 이력

CHANGELOG.md

```
# 사용자 요청
"최근 release v2.3.0의 changelog를 작성해 줘"

# docs-writer의 작업
1. Bash git log v2.2.0..v2.3.0 --oneline
2. 커밋 메시지를 카테고리별 분류 (feat, fix, docs, refactor)
3. 사용자 영향 중심으로 재작성
4. CHANGELOG.md에 추가

# 결과: CHANGELOG.md

## [2.3.0] - 2026-05-25

### Added
- 사용자 API에 페이지네이션 지원 (page, limit 쿼리)
- WebSocket 기반 실시간 알림 채널
- Rate limit 헤더 응답 추가

### Fixed
- /users 엔드포인트에서 deleted 사용자 노출 버그
- JWT 토큰 만료 시 401 대신 500 반환 문제

### Changed
- API 응답 형식 통일 (data/errors envelope)
- 최소 Node 버전 18로 상향
```

git log
커밋 분석

Category
feat/fix/...

User-focus
사용자 영향 중심

SemVer
버전 규칙

CI 통합

PR 시 문서 자동 동기화

```
# .github/workflows/docs-sync.yml
name: Documentation Sync
on:
  pull_request:
    paths:
      - 'src/**/*.js,ts'

jobs:
  sync-docs:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Sync docs
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "@docs-writer 이번 PR의 코드 변경에 맞춰
          관련 문서를 docs/ 디렉토리에서 업데이트해 주세요."

      - name: Commit docs
        run: |
          git config user.email "ci@example.com"
          git add docs/
          git diff --staged --quiet || git commit -m "docs: sync"
          git push
```

docs-sync.yml

PR trigger

PR마다 자동

Headless

-p 모드

Auto commit

문서 커밋

Sync

코드와 일치

문서 스타일 준수

팀 표준에 맞추는 방법

METHOD 1

CLAUDE.md 명시

프로젝트 CLAUDE.md에 문서 톤, 용어, 형식 규칙을 명시.

METHOD 2

기존 문서 학습

docs-writer가 기존 문서를 먼저 Read하여 스타일을 파악.

METHOD 3

Glossary 파일

docs/glossary.md에 표준 용어집을 두고 매번 참조.

METHOD 4

Pre-commit 검증

markdownlint나 textlint로 형식 자동 검증, 위반 시 차단.

EXAMPLE / CLAUDE.md 문서 규칙 섹션

용어: "사용자(user)" 통일 — "유저", "회원" 사용 금지. 톤: 정중한 평어. 코드 예시는 반드시 실행 가능. 영어 용어는 처음 등장 시 한국어 병기.

CLAUDE.md

규칙 명시

Glossary

용어집

Lint

자동 검증

Consistent

일관성 확보

Part 7 Recap

Docs Writer 핵심 정리

TAKEAWAY 1

4가지 문서 종류

README, API 레퍼런스,
ADR, Changelog를 자동
생성/갱신

TAKEAWAY 2

코드가 진실의 원천

문서는 항상 코드를 따르도
록 동기화, CI로 자동 보장

TAKEAWAY 3

스타일 일관성

CLAUDE.md에 규칙 명시,
기존 문서 학습, lint로 검증

TAKEAWAY 4

다국어 지원

한국어/영어 동시 관리, 용
어집으로 일관성 유지

NEXT - PART 8

Part 8은 Pattern 5 Migration입니다. 대량 파일 일괄 변경, 라이브러리 업그레이드 같은 반복 작업을 8장에 걸쳐 다룹니다.

Part 7

완료

Part 8

Migration (8p)

Part 9

Recap & Labs (10p)

~50 min

잔여 시간

CHAPTER 02 – PART 08

Agents – Migration Bot

Subagents 시스템과 실전 패턴



Part 8 / Pattern 5 - Migration Bot

대량 파일 일괄 변환

PATTERN OVERVIEW

Migration Bot은 라이브러리 업그레이드, API 변경, 코드 스타일 통일 같은 대량의 반복적 변경을 자동화하는 에이전트.

USE CASES

사용 사례

lib 업그레이드, deprecated
API 교체, import 경로 변경,
타입 추가

MODEL

모델 선택

Haiku 권장. 단순 반복 작업
이라 비용 효율이 가장 중요.

STRATEGY

진행 전략

소량 시범 → 결과 확인 →
전체 적용. 절대 한 번에 전
체 실행 금지.

SAFETY

안전 장치

Git 브랜치 분리, 단위 테스
트 자동 실행, 결과 diff 사용
자 검토 필수.

8 Slides

Part 8 분량

~15 min

예상 강의 시간

Haiku

권장 모델

Incremental

점진 진행

migration-bot 에이전트 정의

완성된 정의 파일

migration-bot.md

```
---
name: migration-bot
description: |
  대량 파일에서 패턴을 찾아 일관된 규칙으로 변경. 사용 시점: 라이브러리
  업그레이드, deprecated API 교체, import 경로 일괄 변경.
tools: Read, Grep, Glob, Edit, Bash(npm test:*), Bash(git status)
model: claude-haiku-4-5
---
```

당신은 코드 마이그레이션 전문가입니다.

작업 절차

1. 변경 대상 파일을 Glob으로 정확히 식별
2. 5개 파일을 먼저 시범 변경
3. 단위 테스트 실행해 통과 확인
4. 사용자에게 결과 보고 후 전체 진행 동의 요청
5. 동의 시 나머지 파일 일괄 변경

작성 원칙

- 변경은 반드시 패턴 기반, ad-hoc 수정 금지
- 매 5개 파일마다 테스트 실행
- 실패 시 즉시 중단하고 보고
- 절대 main 브랜치에 직접 적용하지 않음

Haiku

경량 모델

Pilot 5

시범 5개 우선

Test

매번 테스트

Branch

브랜치 격리

시나리오 1 / 라이브러리 업그레이드

axios v0.x → v1.x 마이그레이션

axios upgrade

```
# 사용자 요청  
"axios를 v0.27에서 v1.6으로 업그레이드하고 변경 사항을 모두 적용해 줘"
```

```
# migration-bot의 작업
```

```
1. package.json 업데이트: axios ^1.6.0
```

```
2. Grep "import axios" → 23개 파일 식별
```

```
3. 변경 패턴 인식:
```

```
- axios.create(config) → axios.create(config) (변화 없음)
```

```
- response.data.error 처리 변경
```

```
- timeout 옵션 동작 변화
```

```
4. 시범 5개 파일 변경 → npm test 통과
```

```
5. 사용자 검토용 git diff 출력
```

```
6. 동의 → 나머지 18개 파일 변경
```

```
7. 최종 npm test 전체 통과 → 보고
```

```
# 결과 보고
```

```
"23개 파일 변경 완료. 모든 테스트 통과."
```

```
PR 생성 권장: feat/upgrade-axios-v1"
```

23 files

대상 파일

Pattern

패턴 기반

Test

단계별 검증

Report

결과 보고

시나리오 2 / Import 경로 변경

lodash → lodash-es 전환

lodash migration

시나리오: 트리 세이킹을 위해 lodash 전체 import를 lodash-es로 변경

변경 전

```
import _ from 'lodash';  
const result = _.debounce(fn, 300);
```

변경 후 (가능한 경우 named import 우선)

```
import { debounce } from 'lodash-es';  
const result = debounce(fn, 300);
```

migration-bot의 진행

1. Glob "**/*.js,ts,tsx" → 187개 파일
2. Grep "import _ from 'lodash'" → 42개 매칭
3. 각 파일의 _.xxx 사용 추출 → named import로 재작성
4. package.json: lodash → lodash-es 교체
5. 시범 5개 → 빌드 + 테스트 통과
6. 나머지 일괄 진행 → 최종 검증
7. 번들 크기 변화 측정 보고

187

전체 스캔

42

대상 매칭

Tree-shake

목표 효과

Bundle

크기 측정

시나리오 3 / Deprecated API 교체

Node fetch transition

fetch transition

시나리오: node-fetch 외부 라이브러리 제거, Node 내장 fetch로 전환

변경 전

```
const fetch = require('node-fetch');
const res = await fetch(url);
const data = await res.json();
```

변경 후 (Node 18+ 내장 fetch)

```
// require 제거, 내장 fetch 사용
const res = await fetch(url);
const data = await res.json();
```

migration-bot의 차이점 처리

- node-fetch v2: cjs, AbortController 폴리필 필요
- 내장 fetch: cjs/esm, AbortController 내장
- 일부 옵션 비호환:
 - agent → undici Agent (별도 처리 필요)
 - timeout → AbortSignal.timeout()로 대체

작업

1. node-fetch require 제거 (88개 파일)
2. 옵션 호환성 매핑 적용
3. .nvmrc 확인 → Node 18+ 보장
4. 테스트 + 통합 테스트 통과 확인

node-fetch

대체 대상

Native

내장 fetch

Compat

옵션 매핑

Node 18+

필수 버전

안전 장치

Migration의 5가지 안전 메커니즘

SAFETY 1

브랜치 격리

feat/migration-* 브랜치에서만 작업, main 직접 변경 금지

SAFETY 2

시범 후 확장

5개 파일 시범 → 테스트 통과 → 사용자 동의 → 나머지 진행

SAFETY 3

단위 테스트 강제

각 단계에서 npm test 통과 필수, 실패 시 즉시 중단

SAFETY 4

Diff 검토

git diff로 변경 내용을 사용자에게 보여주고 확인 받음

SAFETY 5 / 롤백 준비

항상 변경 전 커밋을 명확히 두어 git revert로 즉시 되돌릴 수 있도록 합니다. 마이그레이션 PR은 가능한 한 작게 쪼개 review와 revert를 용이하게 합니다.

Branch
격리

Pilot
시범

Test
검증

Revert
롤백

비용 효율

왜 Haiku인가

cost calc

시나리오: 200개 파일에서 동일한 패턴 변경

Sonnet 사용 시

- 파일당 input 2K + output 0.5K
- 비용 = $200 \times (2K \times \$3 + 0.5K \times \$15) / 1M$
- = $200 \times \$0.0135 = \2.70

Haiku 사용 시

- 같은 작업, 같은 토큰 사용량
- 비용 = $200 \times (2K \times \$1 + 0.5K \times \$5) / 1M$
- = $200 \times \$0.0045 = \0.90

절감 효과: $\$2.70 \rightarrow \0.90 (-67%)

품질 영향

- 단순 패턴 매칭/교체 작업
- 컨텍스트 이해 필요 없음
- Haiku로도 충분한 품질
- 복잡한 추론이 필요한 작업이 아니라면 Haiku 적극 권장

-67%
비용 절감

Haiku
권장 모델

Pattern
단순 패턴 작업

Bulk
대량 처리

Part 8 Recap

Migration Bot 핵심 정리

TAKEAWAY 1

5가지 사용 사례

lib 업그레이드, API 교체,
import 변경, 타입 추가, 스
타일 통일

TAKEAWAY 2

Haiku 권장

단순 반복 작업, 67% 비용
절감 효과, 품질 영향 없음

TAKEAWAY 3

점진 진행 필수

5개 시범 → 검증 → 동의 →
확장. 한 번에 전체 금지

TAKEAWAY 4

5중 안전 장치

브랜치/시범/테스트/diff/
롤백 모두 준비된 후에만 진
행

NEXT - PART 9

마지막 Part 9입니다. Chapter 2 전체 핵심 요약과 4가지 실습 랩 가이드를 10장에 걸쳐 다룹니다.

Part 8

완료

Part 9

Recap & Labs (10p)

4 Labs

Chapter 2 실습

~25 min

잔여 시간

CHAPTER 02 – PART 08

Agents – 요약 및 Hands on LAB

Subagents 시스템과 실전 패턴



Chapter 2 핵심 요약

9개 Part 100장의 압축본

CONCEPT

개념

독립 격리, 4대 가치, 6대 사용 사례, 적절한 모델 선택

DEFINITION

정의

.claude/agents, YAML frontmatter 4필드, 시스템 프롬프트 4섹션

DISPATCH

Task 도구

자동/수동 호출, 병렬/직렬 패턴, prompt 자족성, 결과 처리

PATTERNS

5대 실전 패턴

Code Reviewer, Tester, Security, Docs, Migration

OUTCOME

이제 Subagent의 개념, 정의 방법, 디스패치 메커니즘, 그리고 5가지 실전 패턴을 모두 익혔습니다.
본인 프로젝트에 맞춰 Subagent를 작성하고 운영할 수 있게 됩니다.

9 Parts

완료

100 Slides

Chapter 2

~3.5h

강의 시간

4 Labs

실습 시작

Hands-on Labs / 4종

직접 손에 익히는 시간

| Lab | 내용 | 예상 시간 |
|--------------------|-----------------------------|-------|
| Lab 1 / 첫 Subagent | code-reviewer 에이전트 작성과 첫 호출 | 20분 |
| Lab 2 / Tools 격리 | 권한 화이트리스트 적용과 검증 | 15분 |
| Lab 3 / Task 디스패치 | 자동, 수동, 병렬 호출 패턴 실습 | 25분 |
| Lab 4 / 멀티 협업 | 여러 에이전트로 종합 점검 워크플로 구현 | 30분 |

TOTAL

4개 랩 합산 약 90분. 본인 프로젝트로 진행하면 더 의미 있습니다. 어려움이 있을 때 즉시 멘토에게 도움 요청 가능.

4 Labs

실습 수

~90 min

총 시간

Own repo

본인 프로젝트

Mentor

도움 가능

Lab 1 / 첫 Subagent

code-reviewer 작성과 호출 (20분)

STEP 01

디렉토리
생성

STEP 02

에이전트
파일 작성

STEP 03

/agents로
등록 확인

STEP 04

PR로
첫 호출

STEP 05

결과
검토

```
# 1. 디렉토리 생성
$ mkdir -p .claude/agents
```

```
# 2. code-reviewer.md 작성 (Part 2 슬라이드 31 참고)
$ vim .claude/agents/code-reviewer.md
```

```
# 3. 등록 확인
$ claude
> /agents
code-reviewer (PR diff 검토)
```

```
# 4. 첫 호출
> 다음 git diff를 검토해 주세요: <diff>...</diff>
```

```
# 5. 결과 검토
- 출력 형식이 적절한가
- 우선순위가 명확한가
- 본인 프로젝트 컨벤션을 따르는가
```

Lab 1 steps

20 min

예상 소요

.claude/agents

저장 위치

/agents

등록 확인

First call

첫 호출

Lab 2 / Tools 권한 격리

안전한 에이전트 만들기 (15분)

Lab 2

1. 의도적으로 위험한 도구 사용을 시도하는 에이전트 작성

```
# .claude/agents/dangerous-test.md
---
name: dangerous-test
description: 의도적 권한 테스트용 에이전트
tools: Read, Grep
model: claude-haiku-4-5
---
```

당신은 테스트 에이전트입니다.
파일을 분석하고 결과를 보고하세요.

2. 일부러 권한을 벗어난 요청
> @dangerous-test src/db.js의 비밀번호 부분을 수정해 줘

3. 결과 관찰
- Edit 도구가 tools에 없음 → 거부됨
- "권한 부족" 메시지로 안전 동작 확인

4. tools 필드 수정 → Edit 추가 → 재시도
의도적인 권한 부여 vs 자동 모든 권한의 차이 체험

15 min

예상 소요

tools

권한 필드

Reject

거부 확인

Explicit

명시 권한

Lab 3 / Task 디스패치

3가지 호출 패턴 실습 (25분)

Lab 3

시나리오: 사전에 code-reviewer, test-writer, docs-writer 모두 등록되어 있음

패턴 A: 자동 디스패치

\$ claude

> 이 PR을 코드 리뷰해 줘

메인인 description 보고 code-reviewer 자동 선택 확인

패턴 B: 수동 명시 호출

> @test-writer src/payment.js의 테스트 커버리지를 90%로 올려 줘

메인인 다른 에이전트 검토 없이 test-writer 호출

패턴 C: 병렬 호출

> 이 코드베이스를 종합 점검해 줘

- 보안 이슈 점검

- 테스트 커버리지 분석

- 문서 일지성 검토

메인인 3개 Subagent를 동시 호출하는 것 관찰

4. /cost로 비용 비교

패턴 A: 단일 → 약 \$0.05

패턴 C: 3개 병렬 → 약 \$0.18 (3배, 시간은 1.5배)

25 min

예상 소요

3 patterns

호출 패턴

Cost compare

비용 비교

/cost

확인 명령

Lab 4 / 멀티 에이전트 협업

종합 점검 워크플로 구현 (30분)

Lab 4 workflow

```
# 시나리오: PR 자동 검증 워크플로
# .github/workflows/pr-check.yml
name: PR Check with Subagents
on: pull_request
jobs:
  comprehensive-check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g @anthropic-ai/claude-code
      # 직렬: code-reviewer 먼저 → 결과 보고
      - name: Code Review
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          claude -p "@code-reviewer git diff main..HEAD" \
            --output-format text > review.md
      # 병렬: 테스트와 보안 동시
      - name: Tests + Security
        run: |
          claude -p "다음은 모두 점검해 줘: 1) 테스트 커버리지 부족
            2) 보안 이슈" --output-format json > result.json
      # 결과를 PR 코멘트로 게시
      - uses: peter-evans/create-pr-comment@v1
        with: { body-path: review.md }
```

30 min

예상 소요

Serial+Parallel

복합 패턴

PR comment

결과 게시

Production

실무 패턴

자주 묻는 질문

Chapter 2 관련 FAQ

FAQ

Q1. Subagent를 너무 많이 만들면 어떤 문제가 있나요?

A. 메인인 description 매칭으로 후보를 추리므로 너무 많으면 매칭이 헛갈립니다. 5~10개 이내가 적정. description을 명확히 작성하세요.

Q2. Subagent가 또 다른 Subagent를 호출할 수 있나요?

A. 기술적으로 가능하지만 권장하지 않습니다. 복잡도가 빠르게 증가하고 디버깅이 어려워집니다. 메인에서 직렬 호출이 더 명확합니다.

Q3. 팀 전체에 Subagent를 어떻게 배포하나요?

A. .claude/agents를 Git에 커밋하면 됩니다. PR 리뷰로 품질 관리도 가능.

Q4. Subagent가 실수로 위험한 변경을 했을 때 복구는?

A. Git이 가장 강력한 안전망. 항상 브랜치에서 작업하고 PR로 검토.

Q5. Subagent 비용이 너무 큼니다. 어떻게 줄이나요?

A. (1) 단순 작업은 Haiku (2) /cost로 모니터링 (3) tools 좁히기
(4) Opus는 정말 필요할 때만 (5) 결과 캐싱 패턴 도입

5+ Q

대표 질문

Doc

공식 문서

Slack

사내 채널

Practice

직접 경험

Next Chapter Preview

Chapter 3 / Admin Setup 예고

CHAPTER 3 OVERVIEW

Day 2 오전에 진행될 Chapter 3은 Admin Setup입니다. Enterprise 환경에 Claude Code를 안전하게 배포하는 모든 절차를 다룹니다.

TOPIC 1

사내 배포 전략

npm 미러, 사내 패키지 레지스트리, MSI 대량 배포

TOPIC 2

자격증명 관리

AWS Bedrock, Vertex AI, SSO 통합, IAM 정책 설계

TOPIC 3

네트워크와 보안

프록시 통과, 인증서, 화이트리스트, 사내 SIEM 통합

TOPIC 4

거버넌스

조직 정책, 사용량 모니터링, 비용 통제, 감사 로그

Day 2 AM

진행 시간

120 Slides

Chapter 3 분량

Enterprise

대상 환경

Admin

역할 대상

References

참고 자료 및 출처

REFERENCES

Official Documentation

<https://code.claude.com/docs/ko/agents>

<https://code.claude.com/docs/ko/task-tool>

<https://code.claude.com/docs/ko/agent-permissions>

Example Agents (GitHub)

<https://github.com/anthropics/claude-code-examples>

<https://github.com/anthropics/awesome-claude-agents>

Patterns and Best Practices

<https://docs.anthropic.com/claude-code/patterns>

<https://docs.anthropic.com/claude-code/cost-optimization>

Related Tools

Model Context Protocol: <https://modelcontextprotocol.io>

GitHub Actions Marketplace: [actions/setup-node](#), [peter-evans/*](#)

Community

Anthropic Discord: discord.gg/anthropic

Reddit: [r/ClaudeAI](#)

Docs

공식 문서

GitHub

예제 저장소

Patterns

베스트 프랙티스

Community

커뮤니티

Thank you!

Choi, WooHyung / Prin. Solutions Architect / AWS Korea

whchoi@amazon.com

<https://linkedin.com/in/woohyungchoi>

© 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Amazon Confidential and Trademark.

